

Check if a Linked List is a Palindrome

Personal DSA Study Notes — Java

1. Introduction

A palindrome remains the same when its order is reversed. For a linked list, we compare values from the beginning with values from the end. Because a singly linked list cannot move backward, the optimized method temporarily reverses the second half.

2. Problem Statement

Given the head of a singly linked list, return true if it is a palindrome; otherwise return false.

List	Result
1 → 2 → 3 → 2 → 1	true
1 → 2 → 3 → 2 → 3	false
1 → 2 → 2 → 1	true

3. Required Concepts

- Linked-list traversal
- Stack / LIFO
- Slow and fast pointers
- Linked-list reversal
- Comparing two halves
- Restoring the original list

4. Approach 1 — Stack

Store every value in a stack. Since a stack is LIFO, popping gives the values in reverse order. Compare those values with the original list.

Pseudocode

```
stack = empty stack
temp = head

while temp != null:
    stack.push(temp.val)
    temp = temp.next

temp = head

while temp != null:
    if temp.val != stack.top():
        return false
    temp = temp.next
    stack.pop()

return true
```

Complete Java Code

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        Stack<Integer> stack = new Stack<>();
        ListNode temp = head;

        while (temp != null) {
            stack.push(temp.val);
            temp = temp.next;
        }

        temp = head;
```

```

        while (temp != null) {
            if (temp.val != stack.peek()) {
                return false;
            }
            temp = temp.next;
            stack.pop();
        }

        return true;
    }
}

```

Complexity: $O(n)$ time and $O(n)$ extra space.

5. Approach 2 — Reverse the Second Half

The optimized method has three main parts: find the first-half boundary, reverse the second half, and compare both halves. Finally, reverse the second half again to restore the original list.

Part 1 — Find the Boundary

```

slow = head
fast = head

while fast.next != null AND fast.next.next != null:
    slow = slow.next
    fast = fast.next.next

second half = slow.next

```

For an even-length list, slow stops at the first middle. For an odd-length list, slow stops at the middle.

Part 2 — Recursive Reverse

```

private ListNode reverseLinkedList(ListNode head) {
    if (head == null || head.next == null) {
        return head;
    }

    ListNode newHead = reverseLinkedList(head.next);
    ListNode front = head.next;
    front.next = head;
    head.next = null;

    return newHead;
}

```

Static Visual

```

Before:  1 → 2 → 3 | 2 → 1 → NULL
After:   1 → 2 → 3 | 1 → 2 → NULL
                ↑
            newHead

```

Part 3 — Compare

```

first = head
second = newHead

while second != null:
    if first.val != second.val:
        restore second half
        return false

    first = first.next
    second = second.next

restore second half
return true

```

6. Complete Optimized Java Code

```

class Solution {

    private ListNode reverseLinkedList(ListNode head) {
        if (head == null || head.next == null) {

```

```

        return head;
    }

    ListNode newHead = reverseLinkedList(head.next);
    ListNode front = head.next;
    front.next = head;
    head.next = null;

    return newHead;
}

public boolean isPalindrome(ListNode head) {
    if (head == null || head.next == null) {
        return true;
    }

    ListNode slow = head;
    ListNode fast = head;

    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    ListNode newHead = reverseLinkedList(slow.next);

    ListNode first = head;
    ListNode second = newHead;

    while (second != null) {
        if (first.val != second.val) {
            reverseLinkedList(newHead);
            return false;
        }

        first = first.next;
        second = second.next;
    }

    reverseLinkedList(newHead);
    return true;
}
}

```

7. Static Algorithm Visual

1 → 2 → 3 → 2 → 1

Find boundary:

1 → 2 → 3 | 2 → 1

Reverse second half:

1 → 2 → 3 | 1 → 2

Compare:

1 ↔ 1

2 ↔ 2

3 ↔ 2

...

Restore:

reverse the second half again

8. Edge Cases

- Empty list → true
- One node → true
- Two equal nodes → true
- Two different nodes → false
- Odd palindrome → true
- Even palindrome → true
- Any mismatch → false

9. Why Restore the List?

The second half is reversed only temporarily. Before returning true or false, reverse it again so the input linked list returns to its original structure.

10. Complexity

The stack approach uses $O(n)$ extra space. The optimized pointer approach performs $O(n)$ total work. Because the reverse function here is recursive, its call stack uses $O(n)$ auxiliary space. The linked-list pointer storage itself is $O(1)$.

11. Quick Revision

1. Handle null / one node.
2. Find the first-half boundary.
3. Reverse `slow.next`.
4. Compare head with the reversed second half.
5. On mismatch: restore and return false.
6. If all match: restore and return true.

Key idea:
Singly list cannot move backward,
so reverse the second half temporarily.

12. Conclusion

The stack solution is straightforward but needs $O(n)$ extra memory. The optimized method uses the linked list itself: locate the boundary, reverse the second half, compare, and restore. With the recursive reverse helper, auxiliary recursion space is $O(n)$.